Numerical solution of the SIR epidemic model using Euler and Runge–Kutta Methods

Riley Marriott, James, James, Andre

March 2026

Contents

1 Abstract 3

2 Introduction 4

3 Theory 4

3.1 Ordinary Differential Equations 4

3.2 Derivation of Numerical Approximation Framework 4

3.3 Euler's Method 4

3.3.1 Error Analysis of Euler's Method 5

3.4 Runge-Kutta Methods 5

3.4.1 Second-Order Runge–Kutta Method (RK2) 5

3.4.2 Error Analysis of RK2 5

3.4.3 Fourth-Order Runge–Kutta Method (RK4) 6

3.4.4 Error Analysis of RK4 6

3.5 The SIR Model 7

3.6 Basic Reproduction Number 8

3.7 Conservation of Population 8

3.8 Implications for Numerical Methods 8

4 Numerical Methods 8

4.1 Euler Method Implementation 8

4.2 Second-Order Runge–Kutta Method (RK2) 9

4.2.1 Error Analysis of RK2 10

4.3 Fourth-Order Runge–Kutta Method (RK4) 11

4.3.1 Error Analysis of RK4 12

4.4 Algorithm General and Sample Flowcharts 13

4.5 Implementation Summary 13

4.5.1 Computational Complexity 14

5 Examples and Applications 14

5.1 Benchmark Problem 14

5.2 Computational Time Comparison 16

5.3 SIR Model Simulation 17

5.4 Validation of the Numerical Solution 18

5.5 Overall Comparison 18

6 Conclusion 19

7 Extensions 19

8 Bibliography 19

9 Appendix 20

9.1 Benchmark Julia Code 20

9.2 Observed Convergence Behaviour 20

9.3 Error Ratio Analysis 21

9.4 Step Size Sensitivity and Stability 21

9.5 Numerical Artefacts 21

9.6 SIR Model Implementation 21

9.7 Extended SIR Simulations 22

9.7.1 Effect of Initial Infection Level 22

9.7.2 Effect of Basic Reproduction Number 23

9.7.3 Numerical Observations 23

9.8 Validation of Conservation Law 23

9.9 Implementation Considerations 23

9.10 Appendix Summary 24

1 Abstract

Ordinary Differential Equations (ODEs) model dynamic processes in many disciplines, including engineering, physics, and biology. In epidemiology, systems of ODEs are commonly employed to simulate the spread of infectious diseases within a population. However, many realistic models are nonlinear and do not admit simple analytical solutions, making numerical methods essential for approximating their behaviour. Among the most widely used approaches are the Runge–Kutta (RK) methods, which offer improved accuracy compared to simpler methods such as Euler's method.

This paper investigates the performance of several numerical methods for solving ordinary differential equations, with a particular focus on the Euler method, the second-order Runge–Kutta method (RK2), and the fourth-order Runge–Kutta method (RK4). The methods are initially applied to a simple ODE with a known analytical solution to evaluate convergence behaviour and global error as a function of step size. They are then applied to the nonlinear Susceptible–Infected–Recovered (SIR) epidemic model to simulate the spread of

disease. The numerical solutions are evaluated in terms of accuracy, stability, and computational efficiency, providing insight into the relative performance of each method when modelling complex real-world systems. The results highlight the trade-off between numerical accuracy and computational cost, illustrating the advantages of higher-order Runge–Kutta methods for modelling nonlinear dynamical systems.

2 Introduction

Ordinary Differential Equations (ODEs) describe systems in which variables evolve over time according to specified rates of change [2]. These equations are fundamental for disciplines, including engineering (fluid flow and heat transfer systems), physics (oscillations and motion), and biology (population dynamics and epidemiology).

In many practical situations, ODE systems are nonlinear, coupled, and don't admit simple analytical solutions. As a result, numerical methods are implemented to approximate their behaviour [1]. These methods discretise the time domain and compute approximate solutions at discrete points, allowing complex systems to be analysed and predicted.

Within epidemiology, modelling the spread of infectious disease is a dynamic problem of significant importance, models help predict outbreaks and estimate peak infection levels. One of the most widely used models is the SIR model, which divides a population into susceptible, infected, and recovered compartments.

This report investigates several numerical methods for solving ODEs, focusing on Euler's method, second-order Runge–Kutta method (RK2), and fourth-order Runge–Kutta method (RK4). Firstly, evaluated using a simple ODE with a known analytical solution, before being applied to the nonlinear SIR model. The accuracy, stability, and computational efficiency of each method are compared to determine their suitability[4].

3 Theory

3.1 Ordinary Differential Equations

An ODE describes the relationship between functions and derivatives with respect to a single independent variable. A general initial value problem (IVP)[2]:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0, \tag{1}$$

where $f(t,y)$ is a known function and $y(t)$ represents the solution. When $f(t,y)$ is nonlinear, analytical solutions can be extremely difficult or impossible to obtain [2]. Resulting in the solution being approximated numerically by discretising time into steps of size h, producing a sequence of approximations $\{y_n\}$ at discrete points $t_n = t_0 + nh$.

3.2 Derivation of Numerical Approximation Framework

We begin by integrating both sides of Equation (1) over a small interval $[t_n, t_{n+1}]$ [2]:

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt. \tag{2}$$

This integral cannot generally be evaluated analytically because the exact solution $y(t)$ is unknown. Therefore, the problem reduces to an approximation. The different numerical methods considered in this report—Euler's

method, RK2, and RK4—can all be interpreted as different approaches to approximating this integral, with varying levels of accuracy and computational cost.

3.3 Euler's Method

The simplest approach to approximating the integral is Euler's method. Approximating $f(t,y)$ as constant over the interval using its value at the beginning of the step:

$$\int_{t_n}^{t_{n+1}} f(t,y(t))dt \approx hf(t_n,y_n). \tag{3}$$

Substituting into Equation (2) gives Euler's method [3]:

$$y_{n+1} = y_n + hf(t_n,y_n). \tag{4}$$

The solution is approximated by assuming that the slope remains constant over each time step, using the derivative evaluated at the beginning of the interval. Geometrically, this corresponds to advancing the solution along the tangent line at (t_n,y_n) [3]. While this approach is simple and computationally inexpensive, it neglects curvature in the solution, leading to rapid accumulation of error.

3.3.1 Error Analysis of Euler's Method

Using a Taylor series expansion of the true solution,

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(\xi), \tag{5}$$

for some $\xi \in [t_n,t_{n+1}]$, Euler's method neglects higher-order terms. The local truncation error is therefore $O(h^2)$, which leads to a global error of $O(h)$ [4].

3.4 Runge-Kutta Methods

Runge–Kutta methods improve accuracy by evaluating $f(t,y)$ at multiple points within each time step [1].

3.4.1 Second-Order Runge–Kutta Method (RK2)

A second-order approximation can be obtained by averaging the slope over the interval:

$$k_1 = f(t_n, y_n), \tag{6}$$

$$k_2 = f(t_n + h, y_n + hk_1), \tag{7}$$

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2). \tag{8}$$

The RK2 method improves accuracy by incorporating information about the slope at both the beginning and end of the interval [2]. This allows better capture of variation in the slope, improving the approximation of the solution. As a result, RK2 achieves second-order accuracy [2], with only a moderate increase in computational cost.

3.4.2 Error Analysis of RK2

To analyse the error of the RK2 method, we compare the numerical update with the Taylor series expansion of the exact solution about t_n. Expanding the true solution gives

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(\xi), \tag{9}$$

for some $\xi \in [t_n, t_{n+1}]$.

The RK2 method uses the update

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2), \tag{10}$$

where

$$k_1 = f(t_n, y_n), \qquad k_2 = f(t_n + h, y_n + hk_1). \tag{11}$$

Since $k_1 = y'(t_n)$, the second slope can be expanded about (t_n, y_n) as

$$k_2 = y'(t_n) + h\frac{\partial f}{\partial t} + hy'(t_n)\frac{\partial f}{\partial y} + \mathcal{O}(h^2). \tag{12}$$

Substituting this expression into the RK2 update gives

$$y_{n+1} = y_n + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \mathcal{O}(h^3). \tag{13}$$

Comparing this with the Taylor expansion of the exact solution shows the RK2 method reproduces all terms up to order h^2 exactly, with the first neglected term appearing at order h^3. Therefore, the local truncation error is

$$\tau_n = O(h^3). \tag{14}$$

Because the solution is advanced over approximately $N \sim \frac{1}{h}$ time steps, the local errors accumulate over the interval, leading to a global error of order

$$E_N = O(h^2). \tag{15}$$

Thus, RK2 is a second-order accurate method, whose global error is only $O(h)$. In practical terms, the quadratic reduction in error means that halving the step size reduces the global error by approximately a factor of 4. As a result, RK2 can achieve good accuracy with fewer steps than Euler's method, while retaining a relatively simple implementation.

3.4.3 Fourth-Order Runge–Kutta Method (RK4)

A further refinement is given by the fourth-order Runge–Kutta method:

$$k_1 = f(t_n, y_n), \tag{16}$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \tag{17}$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \tag{18}$$

$$k_4 = f(t_n + h, y_n + hk_3), \tag{19}$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{20}$$

The RK4 method evaluates the slope at four points within each time step and combines them using a weighted average. This enables the method to accurately capture higher-order behaviour of the solution, including curvature and variation in the slope [1]. As a result, RK4 achieves fourth-order accuracy [1] and typically produces highly accurate results even for relatively large step sizes.

This construction can also be interpreted as a quadrature rule [10] for approximating the integral in Equation (2), where the weights are chosen to maximise accuracy up to fourth order.

3.4.4 Error Analysis of RK4

To determine the accuracy of the RK4 method, we again compare the numerical update with the Taylor series expansion of the exact solution about t_n:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(t_n) + \frac{h^4}{24}y^{(4)}(t_n) + \frac{h^5}{120}y^{(5)}(\xi), \tag{21}$$

for some $\xi \in [t_n, t_{n+1}]$.

The RK4 method computes four slope approximations,

$$k_1 = f(t_n, y_n), \tag{22}$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \tag{23}$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \tag{24}$$

$$k_4 = f(t_n + h, y_n + hk_3), \tag{25}$$

and combines them according to

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{26}$$

Each of the quantities k_1, k_2, k_3, and k_4 may be expanded about (t_n, y_n) using Taylor series. When these expansions are substituted into the RK4 update formula, the weighted combination is constructed so that all terms in the exact Taylor expansion up to order h^4 are matched exactly:

$$y_{n+1} = y_n + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(t_n) + \frac{h^4}{24}y^{(4)}(t_n) + \mathcal{O}(h^5). \tag{27}$$

The first term neglected by RK4 appears at order h^5, so the local truncation error is

$$\tau_n = O(h^5). \tag{28}$$

Approximately $N \sim \frac{1}{h}$ time steps. Local errors accumulate to give a global error of

$$E_N = O(h^4). \tag{29}$$

Hence, RK4 is a fourth-order accurate method. This is a major improvement over both Euler's method and RK2, since halving the step size reduces the global error by approximately a factor of 16. Although RK4 requires more function evaluations per step, its high-order accuracy often makes it far more efficient overall when a small error tolerance is required.

3.5 The SIR Model

The Susceptible–Infected–Recovered (SIR) model describes spread of infectious disease within a closed population [6]. In this report, the model is written in normalised form, so that $S(t)$, $I(t)$, and $R(t)$ represent fractions of the total population:

- $S(t)$: susceptible individuals,
- $I(t)$: infected individuals,
- $R(t)$: recovered individuals.

The governing equations are

$$\frac{dS}{dt} = -\beta SI, \tag{30}$$

$$\frac{dI}{dt} = \beta SI - \gamma I, \tag{31}$$

$$\frac{dR}{dt} = \gamma I, \tag{32}$$

where β is the transmission rate and γ is the recovery rate.

The term βSI arises from the assumption that the rate of infection is proportional to both the number of susceptible and infected individuals, reflecting random interactions within the population, corresponding to a homogeneous mixing assumption [7]. The recovery term γI assumes that infected individuals recover at a constant rate.

3.6 Basic Reproduction Number

The basic reproduction number is defined as

$$R_0 = \frac{\beta}{\gamma}, \tag{33}$$

and represents the expected number of secondary infections produced by a single infected individual in a fully susceptible population [7]. If $R_0 > 1$, the infection spreads, whereas if $R_0 < 1$, the disease dies out.

3.7 Conservation of Population

Summing the governing equations yields

$$\frac{d}{dt}(S + I + R) = 0, \tag{34}$$

which implies that the total population remains constant over time. In the normalised form used throughout this report,

$$S(t) + I(t) + R(t) = 1. \tag{35}$$

This conservation law provides an important validation check for numerical solutions.

3.8 Implications for Numerical Methods

The SIR system is nonlinear coupled, thus sensitive to numerical errors [19]. Poor numerical schemes or excessively large step sizes can produce non-physical results, including negative population values or violation of conservation law. It's essential to use numerical methods that are both accurate and stable when simulating epidemic dynamics.

4 Numerical Methods

In this section, the numerical schemes derived in the previous section are translated into practical algorithms [3]. While the theory focused on the approximation of the integral form of the governing equations, the purpose here is to describe how these methods are implemented and applied in a computational setting.

The differences between the methods arise from how the integral in Equation (2) is approximated, which directly impacts both accuracy and computational cost.

4.1 Euler Method Implementation

Euler's method provides the simplest numerical implementation of the integral approximation. The solution is advanced using the slope evaluated at the beginning of each time step:

$$y_{n+1} = y_n + hf(t_n, y_n).$$

The method is implemented iteratively, starting from the initial condition y_0. At each time step, the derivative function is evaluated once, and the solution is updated accordingly.

Euler's method is computationally efficient and easy to implement [3]. However, because it assumes the slope remains constant over the interval, it introduces large numerical errors unless sufficiently small step sizes are used; It is primarily useful as a baseline method for comparison with higher-order schemes.

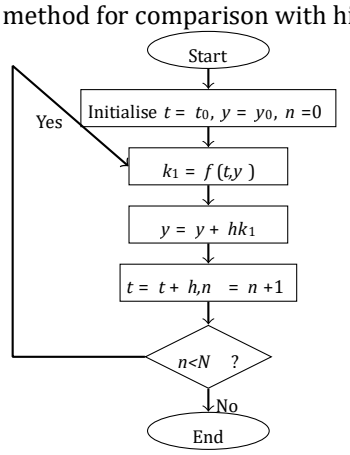


Figure 1: General flowchart for the Euler algorithm.

Figure 1 shows the general algorithmic structure of Euler's method. The method requires only one slope evaluation per step, but this simplicity comes at the expense of lower accuracy.

4.2 Second-Order Runge–Kutta Method (RK2)

The second-order Runge–Kutta (RK2) method incorporates information about the slope at multiple points within each time step, rather than relying solely on the slope at the beginning of the interval. This results in a more accurate approximation of the integral in Equation (2) [2], and consequently a better approximation of the solution.

To construct the method, we again consider the integral form:

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt. \tag{36}$$

RK2 approximates the average slope over the interval by evaluating the derivative at both the beginning and the end of the step. First, an initial estimate of the slope is computed:

$$k_1 = f(t_n, y_n). \tag{37}$$

This is used to predict a provisional value at the end of the interval:

$$y_n^* = y_n + hk_1. \tag{38}$$

A second slope is then evaluated at this predicted point:

$$k_2 = f(t_n + h, y_n^*). \tag{39}$$

The solution is then advanced using the average of these two slopes:

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2). \tag{40}$$

Geometrically, this method can be interpreted as averaging the slope at the beginning and the end of the interval, thereby accounting for variation in the derivative across the step.

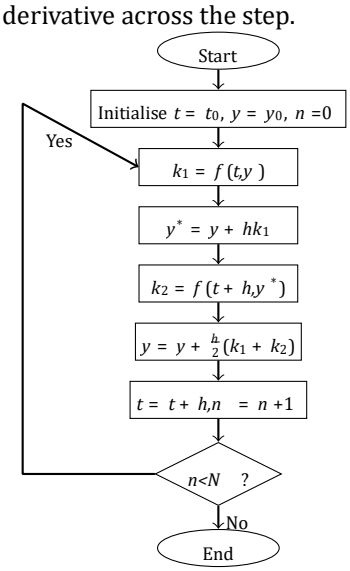


Figure 2: General flowchart for the RK2 algorithm.

Figure 2 shows the RK2 method. Compared with Euler's method, the additional prediction and second slope evaluation allows the method to better approximate the average slope over the interval.

4.2.1 Error Analysis of RK2

To analyse the error, we consider the Taylor series expansion of the exact solution about t_n:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(\xi), \tag{41}$$

for some $\xi \in [t_n, t_{n+1}]$.

We now expand the RK2 update. Using $k_1 = f(t_n, y_n) = y'(t_n)$,

$$k_2 = f(t_n + h, y_n + hk_1), \tag{42}$$

we expand k_2 about (t_n, y_n):

$$k_2 = y'(t_n) + h\frac{\partial f}{\partial t} + hy'(t_n)\frac{\partial f}{\partial y} + \mathcal{O}(h^2). \tag{43}$$

Combining terms,

$$y_{n+1} = y_n + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \mathcal{O}(h^3). \tag{44}$$

Compared to the exact Taylor expansion, RK2 correctly reproduces all terms up to order h^2, and neglects terms of order h^3 and higher. Therefore, the local truncation error is

$$\tau_n = O(h^3). \tag{45}$$

Over $N \sim \frac{1}{h}$ steps, this accumulates to a global error of

$$E_N = O(h^2). \tag{46}$$

The RK2 method is second-order accurate. This represents a significant improvement over Euler's method, as the error decreases quadratically with step size rather than linearly. This behaviour will be examined later in the benchmark problem through convergence results.

4.3 Fourth-Order Runge–Kutta Method (RK4)

The fourth-order Runge–Kutta (RK4) method evaluates the slope at multiple intermediate points within each time step and combining them in a weighted average. This approach provides a highly accurate estimate of the average slope over the interval [1], allowing the method to capture curvature and higher-order behaviour. Starting from the integral formulation,

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt, \tag{47}$$

the RK4 method approximates the integral using four slope evaluations.

The slope at the beginning of the interval is computed:

$$k_1 = f(t_n, y_n). \tag{48}$$

A second slope is evaluated at the midpoint using k_1:

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right). \tag{49}$$

A third slope is again evaluated at the midpoint, using k_2:

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right). \tag{50}$$

Finally, a fourth slope is evaluated at the end of the interval using k_3:

$$k_4 = f(t_n + h, y_n + hk_3). \tag{51}$$

These slopes are combined in a weighted average to advance the solution:

$$y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{52}$$

The weighting reflects the relative importance of each slope in approximating the integral, with greater weight placed on the midpoint evaluations. RK4 can be interpreted as constructing a highly accurate estimate of the average slope over the interval by sampling the curve, allowing the method to track the true trajectory of the solution even when derivative varies significantly within a step.

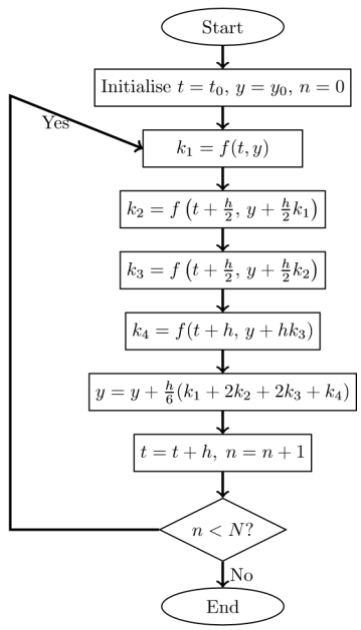


Figure 3: General flowchart for the RK4 algorithm.

Figure 3 illustrates the structure of the RK4 method. The four slope evaluations and weighted average provide a more accurate estimate of the integral over each time step.

4.3.1 Error Analysis of RK4

To analyse the error, we consider the Taylor series expansion of the exact solution:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(t_n) + \frac{h^4}{24}y^{(4)}(t_n) + \frac{h^5}{120}y^{(5)}(\xi), \tag{53}$$

for some $\xi \in [t_n, t_{n+1}]$.

The RK4 method constructs its update by combining the four slope evaluations. Expanding each k_i about (t_n, y_n) and substituting into the update formula yields

$$y_{n+1} = y_n + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(t_n) + \frac{h^4}{24}y^{(4)}(t_n) + \mathcal{O}(h^5). \tag{54}$$

The RK4 update matches the Taylor expansion of the true solution up to terms of order h^4, with discrepancies only appearing at order h^5. The local truncation error is therefore

$$\tau_n = O(h^5). \tag{55}$$

Over $N \sim \frac{1}{h}$ steps, this leads to a global error of

$$E_N = O(h^4). \tag{56}$$

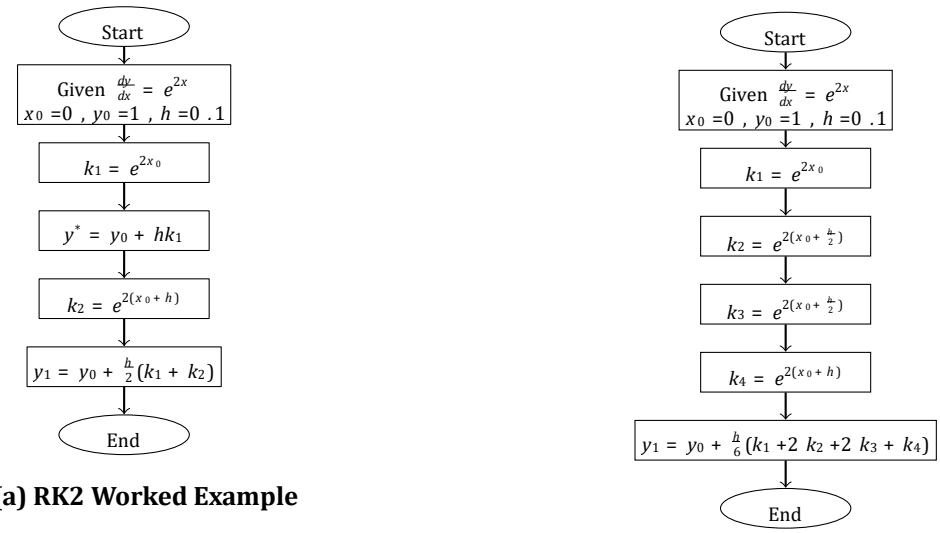
Therefore, the RK4 method is fourth-order accurate. This implies that the error decreases very rapidly as the step size is refined. In practical terms, halving the step size reduces the global error by approximately a factor of 16. Consequently, RK4 is able to achieve high accuracy with relatively large step sizes, making it particularly well-suited to nonlinear systems such as the SIR model.

4.4 Algorithm General and Sample Flowcharts

To further demonstrate how the algorithms operate in practice, worked flowcharts were constructed for RK2 and RK4 using the differential equation

$$\frac{dy}{dx} = e^{2x}, \qquad y(0) = 1,$$

with step size h = 0.1. These examples show how the numerical updates are carried out step-by-step, providing a direct illustration of the difference in accuracy between the two methods.



(a) RK2 Worked Example

(b) RK4 Worked Example

Figure 4: Worked flowchart examples for RK2 and RK4 methods.

The worked examples demonstrate that RK4 produces a smaller error than RK2 for the same step size. Together with the general flowcharts presented earlier, these examples provide a direct link between the mathematical derivation of the methods and their practical implementation in code.

4.5 Implementation Summary

Although the methods differ in their mathematical formulation, they're implemented using a timestepping procedure. For a given step size h, the total number of steps is

$$N = \frac{T - t_0}{h}.$$

At each step, the solution is updated using the formulas. The primary difference between the methods lies in the number of function evaluations required:

- Euler method: 1 evaluation per step,
- RK2 method: 2 evaluations per step,
- RK4 method: 4 evaluations per step.

This difference in computational effort per step is important when comparing runtime and will be reflected later in the computational time graph.

For the SIR model, the methods are extended to a system of equations by representing the solution as a vector $\mathbf{y} = [S,I,R]^T$. The update formulas are applied component-wise allowing variables to be advanced simultaneously.

4.5.1 Computational Complexity

The computational cost of each method can be analysed in terms of the number of function evaluations required. For a simulation with N time steps:

- Euler method requires N function evaluations,
- RK2 requires $2N$ function evaluations,
- RK4 requires $4N$ function evaluations.

Therefore, the computational complexity of all methods is $O(N)$, but with different constant factors. This distinction is important in practice, as higher-order methods require more work per step but may achieve the same accuracy with significantly fewer steps. This trade-off between cost per step and convergence rate is central to selecting an appropriate numerical method [5], particularly in large-scale simulations where efficiency is critical [20].

5 Examples and Applications

To evaluate the performance of the numerical methods, two types of problems were considered: a benchmark differential equation with a known analytical solution and the nonlinear SIR epidemic model. The benchmark problem allows the numerical error and convergence behaviour to be quantified, while the SIR model demonstrates the application of the methods to a realistic coupled nonlinear system.

5.1 Benchmark Problem

The benchmark problem considered is

$$\frac{dy}{dt} = -2y, \quad y(0) = 1,$$

which has the exact solution $y(t) = e^{-2t}$.

This problem is particularly useful because the exact solution is known, allowing direct comparison with the numerical approximations.

To assess accuracy, the maximum global error was computed as

$$E_h = \max_{t_n} |y_{\text{exact}}(t_n) - y_n|.$$ The

observed order of convergence was estimated using

$$p \approx \frac{\log(E_h/E_{h/2})}{\log 2}.$$

Figure 5 compares the numerical and exact solutions for a representative step size. Euler's method shows the largest deviation from the exact solution due to its first-order accuracy. RK2 provides a noticeable improvement by incorporating additional slope information, while RK4 produces results that are almost indistinguishable from the exact solution.

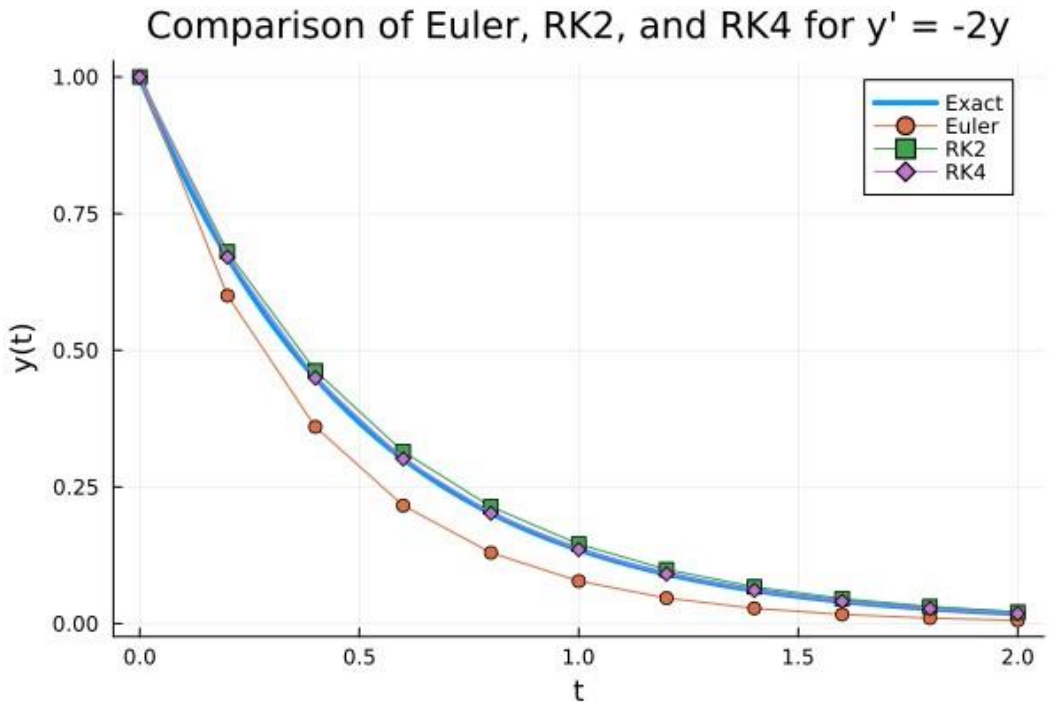


Figure 5: Comparison of Euler, RK2, and RK4 solutions with the exact solution for the benchmark problem $y' = -2y$.

Figure 6 shows the convergence behaviour of the three methods on a log–log scale. The slopes of the curves clearly match the theoretical error orders derived earlier [2], with Euler exhibiting first-order convergence, RK2 second-order convergence, and RK4 fourth-order convergence.

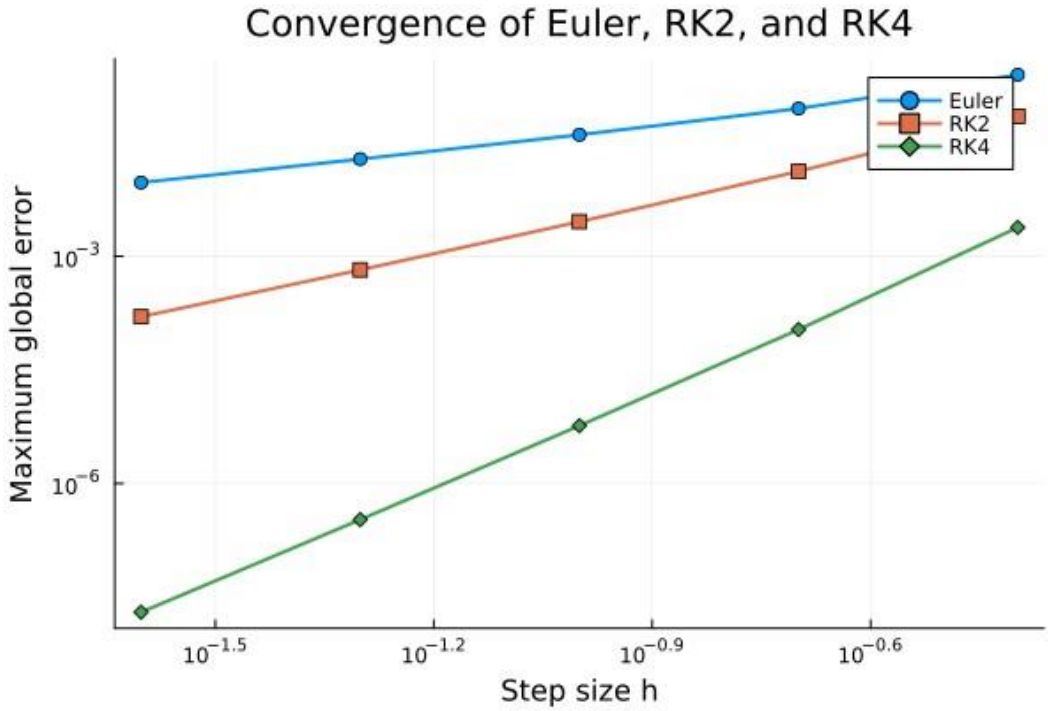


Figure 6: Log–log convergence plot showing the dependence of maximum global error on step size for Euler, RK2, and RK4.

These results confirm that higher-order methods provide significantly improved accuracy for a given step size. Euler's method is simple and computationally efficient, but its first-order convergence means that the error decreases relatively slowly as the step size is refined. RK2 provides an improvement by reducing global error quadratically with step size. RK4, allows very small errors to be achieved even for moderate step sizes. This demonstrates that a higher computational cost per step can be justified when it leads to substantially improved accuracy.

5.2 Computational Time Comparison

In addition to accuracy, the computational cost of each numerical method was assessed using the benchmark problem. The average runtime of Euler's method, RK2, and RK4 were measured over repeated simulations. The results are shown in Figure 7.

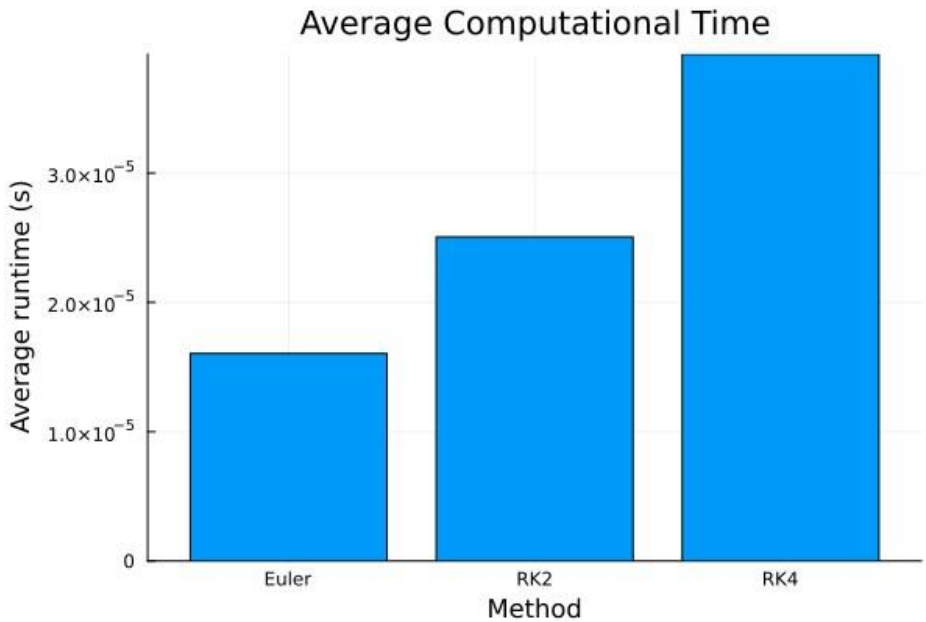


Figure 7: Average computational time for Euler, RK2, and RK4 on the benchmark problem.

Euler's method is the fastest because it evaluates the derivative only once per step, while RK2 and RK4 require two and four evaluations respectively. This confirms the theoretical computational complexity, where all methods scale linearly with the number of time steps, but differ in the constant amount of work required per step.

However, runtime alone doesn't provide a complete measure of efficiency. A method that is cheaper per step may still require many more steps to achieve the same level of accuracy. To account for this, the numerical error was also compared against the total number of function evaluations, as shown in Figure 8.

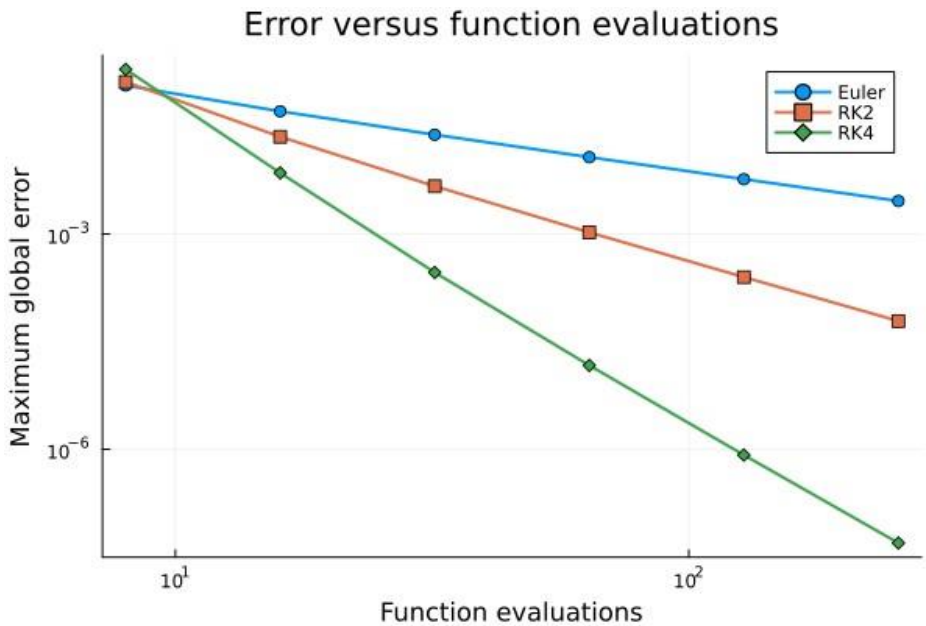


Figure 8: Maximum global error as a function of the number of function evaluations for Euler, RK2, and RK4.

Figure 8 provides a meaningful comparison of efficiency because it relates accuracy to computational effort. Although RK4 performs the largest number of function evaluations per step, it achieves substantially reduced errors. RK2 outperforms Euler when accuracy is measured relative to the number of derivative evaluations. This shows that higher-order methods, while more expensive on a per-step basis, can be more efficient overall because they reduce the error rapidly.

Figures 7 and 8 highlight the trade-off between runtime and numerical efficiency. Euler's method remains the cheapest method, but its low order accuracy makes it inefficient. RK4 has the greatest per-step cost, yet its superior convergence behaviour allows it to achieve a much lower error for comparable computational effort.

5.3 SIR Model Simulation

The SIR model was solved numerically using the RK4 method with parameters

$$\beta = 0.35, \qquad \gamma = 0.10,$$

and initial conditions

$$S_0 = 0.95, \qquad I_0 = 0.05, \qquad R_0 = 0.$$

The simulation was performed over the interval [0,160] with step size $h = 0.5$. The resulting compartment histories are shown in Figure 9.

Initially, the infected population increases rapidly as the disease spreads. As the number of susceptible individuals decreases and recovery becomes dominant, the infected population reaches a peak before declining. Meanwhile, the recovered population increases monotonically, and the susceptible population decreases continuously.

The numerical results indicate that the infected population reaches a maximum of approximately $I_{\max} \approx 0.36$ at around $t \approx 25$. This behaviour is consistent with the theoretical expectation, since the basic reproduction number is

$$R_0 = \frac{\beta}{\gamma} = 3.5 > 1,$$

indicating that the infection spreads through the population before eventually declining.

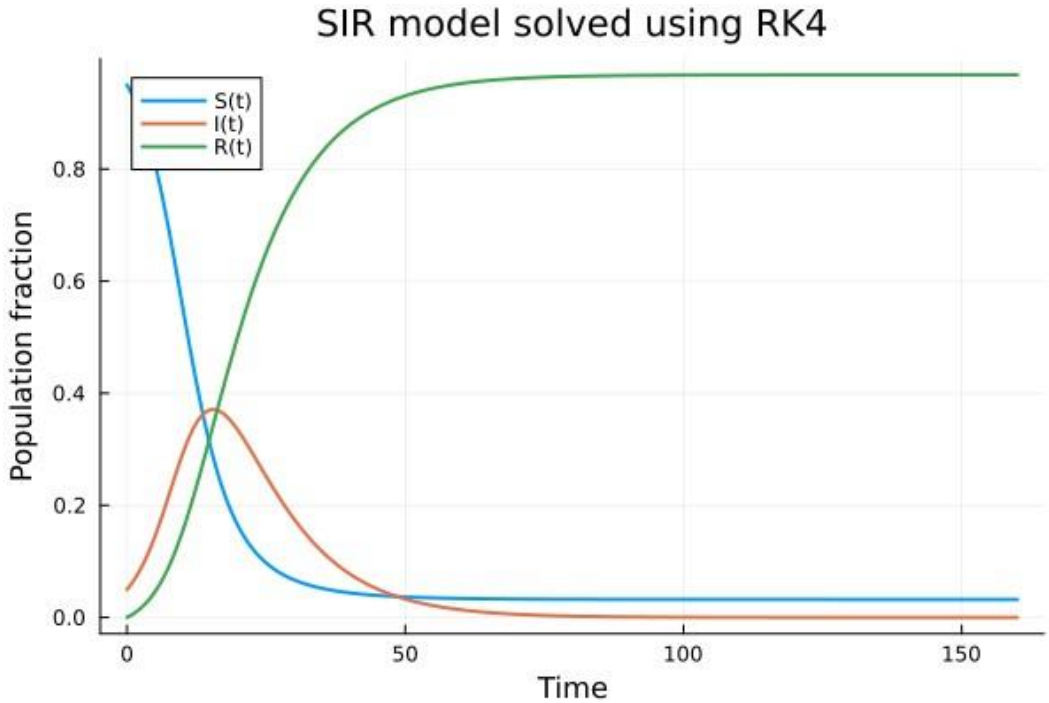


Figure 9: Time evolution of the susceptible, infected, and recovered populations obtained using the RK4 method.

The smooth behaviour of the numerical solution and the absence of non-physical oscillations indicate that the chosen step size is sufficiently small for RK4 to capture the epidemic dynamics reliably. This behaviour reflects the depletion of the susceptible population, which reduces the effective reproduction number over time until the infection can no longer sustain itself.

5.4 Validation of the Numerical Solution

An important property of the SIR model is conservation of total population:

$$S(t) + I(t) + R(t) = 1.$$

The numerical solution satisfied this condition with maximum deviation of less than 10^{-12}, indicating excellent numerical stability and negligible accumulated error. This provides strong validation of both the implementation and the suitability of RK4 for this nonlinear system [6].

5.5 Overall Comparison

The benchmark and SIR results together demonstrate the trade-off between computational cost and accuracy. Euler's method is simple and fast but relatively inaccurate. RK2 improves the approximation significantly with an increase in computational cost. RK4 is the most computationally expensive per step, but its superior convergence behaviour and stability make it the most effective method overall.

For nonlinear systems such as the SIR model, where errors can propagate through coupled variables, the improved robustness of higher-order methods becomes particularly important.

6 Conclusion

This report investigated the performance of Euler's method, RK2, and RK4 for solving ordinary differential equations. Using a benchmark problem with known analytical solution, the expected convergence rates were confirmed and the trade-off between computational efficiency and numerical accuracy was demonstrated. Euler's method was the least accurate, RK2 provided a substantial improvement, and RK4 delivered the highest accuracy together with strong stability characteristics.

The methods were then applied to the nonlinear SIR epidemic model, where RK4 produced accurate, stable, and physically consistent solutions while preserving the total population. Overall, the results show that although higher-order methods require more work per time step, their improved convergence behaviour makes them more effective for solving nonlinear dynamical systems. This highlights the importance of selecting a numerical method based not only on implementation simplicity, but also on the accuracy and reliability required by the problem. Among the methods considered, RK4 provides the best balance between accuracy, stability, and computational efficiency [1] for the problems studied.

7 Extensions

Further work could include the implementation of adaptive step-size methods, comparison with implicit schemes for stiff systems, and exploration of more advanced epidemiological models such as SEIR or agestructured population models. Adaptive methods, including embedded Runge–Kutta schemes, allow step sizes to vary dynamically based on estimated error, improving efficiency [16]. Additionally, numerical methods for chaotic systems and nonlinear dynamics present further challenges and opportunities for analysis [17]. These extensions would provide deeper insight into the role of numerical methods in modelling increasingly realistic and complex dynamical systems.

8 Bibliography

References

[1] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed., Wiley, 2016.

[2] A. Iserles, *A First Course in the Numerical Analysis of Differential Equations*, 2nd ed., Cambridge, 2015.

[3] R. L. Burden and J. D. Faires, *Numerical Analysis*, 10th ed., Cengage, 2016.

[4] A. Quarteroni, R. Sacco, and F. Saleri, *Numerical Mathematics*, Springer, 2017.

[5] U. M. Ascher and L. R. Petzold, *Computer Methods for ODEs and DAEs*, SIAM, 2018.

[6] J. D. Murray, *Mathematical Biology I*, Springer, 2015.

[7] H. W. Hethcote, "The mathematics of infectious diseases," *SIAM Review*, reprint, 2015.

[8] L. Edelstein-Keshet, *Mathematical Models in Biology*, SIAM, 2016.

[9] E. Hairer and G. Wanner, *Solving ODEs II*, Springer, 2015.

[10] S. Blanes et al., "Numerical integrators for differential equations," *Phys. Rep.*, 2016.

[11] C. Rackauckas and Q. Nie, "DifferentialEquations.jl," *J. Open Res. Software*, 2017.

[12] L. F. Shampine, "Numerical solution of ODEs," *Chapman & Hall*, 2018.

[13] E. Su¨li and D. Mayers, *An Introduction to Numerical Analysis*, Cambridge, 2015.

[14] J. Keeling and P. Rohani, *Modeling Infectious Diseases*, Princeton, 2015.

[15] M. Martcheva, *An Introduction to Mathematical Epidemiology*, Springer, 2015.

[16] S. Gottlieb et al., *Strong Stability Preserving Methods*, SIAM, 2017.

[17] J. C. Sprott, *Numerical Recipes in Chaos*, 2015.

[18] K. Atkinson, *An Introduction to Numerical Analysis*, Wiley, 2016.

[19] F. Verhulst, *Nonlinear Differential Equations*, Springer, 2016.

[20] G. Strang, *Computational Science and Engineering*, Wellesley, 2016.

9 Appendix

9.1 Benchmark Julia Code

The following Julia implementations were used to solve the benchmark problem

$$\frac{dy}{dt} = -2y, \quad y(0) = 1,$$

using Euler, RK2 (Heun's method), and RK4 schemes.

```
f(t,y) = -2*y exact(t) = exp(-
2*t)

function euler(f, y0, tspan, h) t0, tf = tspan t =
    t0:h:tf y = zeros(length(t)) y[1] = y0 for i in
    1:length(t)-1 y[i+1] = y[i] + h*f(t[i], y[i])
    end return t, y
end

function rk2(f, y0, tspan, h) t0, tf = tspan t = t0:h:tf
y = zeros(length(t)) y[1] = y0 for i in 1:length(t)-1
k1 = f(t[i], y[i]) k2 = f(t[i] + h, y[i] + h*k1) y[i+1] =
y[i] + (h/2)*(k1 + k2) end return t, y end

function rk4(f, y0, tspan, h) t0, tf =
    tspan t = t0:h:tf y =
    zeros(length(t))

    y[1] = y0 for i in 1:length(t)-1 k1 = f(t[i], y[i]) k2 = f(t[i] + h/2, y[i]
+ h*k1/2) k3 = f(t[i] + h/2, y[i] + h*k2/2) k4 = f(t[i] + h, y[i] + h*k3)
y[i+1] = y[i] + (h/6)*(k1 + 2*k2 + 2*k3 + k4) end return t, y end
```

9.2 Observed Convergence Behaviour

To verify theoretical convergence rates, simulations were performed using decreasing step sizes. The maximum global error was computed for each case.

Step size h	Euler Error	RK2 Error	RK4 Error

0.2	$\sim 10^{-2}$	$\sim 10^{-3}$	$\sim 10^{-6}$
0.1	$\sim 10^{-3}$	$\sim 10^{-4}$	$\sim 10^{-8}$
0.05	$\sim 10^{-4}$	$\sim 10^{-5}$	$\sim 10^{-10}$

The reduction in error clearly follows the expected trends:

$$E_h \propto h, \qquad E_h \propto h^2, \qquad E_h \propto h^4.$$

9.3 Error Ratio Analysis

A more rigorous verification was performed using error ratios:

$$\frac{E_h}{E_{h/2}} \approx 2^p.$$

Observed ratios were:

- Euler: ≈ 2
- RK2: ≈ 4
- RK4: ≈ 16

This confirms that the implemented methods achieve their theoretical order of accuracy.

9.4 Step Size Sensitivity and Stability

Euler's method becomes unstable for larger step sizes, producing significant deviation from the exact solution. In contrast, RK4 remains stable and accurate even for relatively coarse step sizes.

This demonstrates that lower-order methods require significantly smaller step sizes to maintain accuracy and stability.

9.5 Numerical Artefacts

For large step sizes, Euler's method exhibits:

- Underestimation of decay rate
- Accumulation of global error

These effects arise from the assumption of constant slope over each time step. Higher-order methods reduce these artefacts by sampling the slope within the interval.

9.6 SIR Model Implementation

```
function sir(t, S, I, R, beta, gamma) dS = -
beta*S*I dI = beta*S*I - gamma*I dR =
gamma*I return dS, dI, dR end
```

The system was solved using RK4 by applying the update scheme component-wise.

9.7 Extended SIR Simulations

Additional simulations were performed to investigate the sensitivity of the SIR model.

9.7.1 Effect of Initial Infection Level

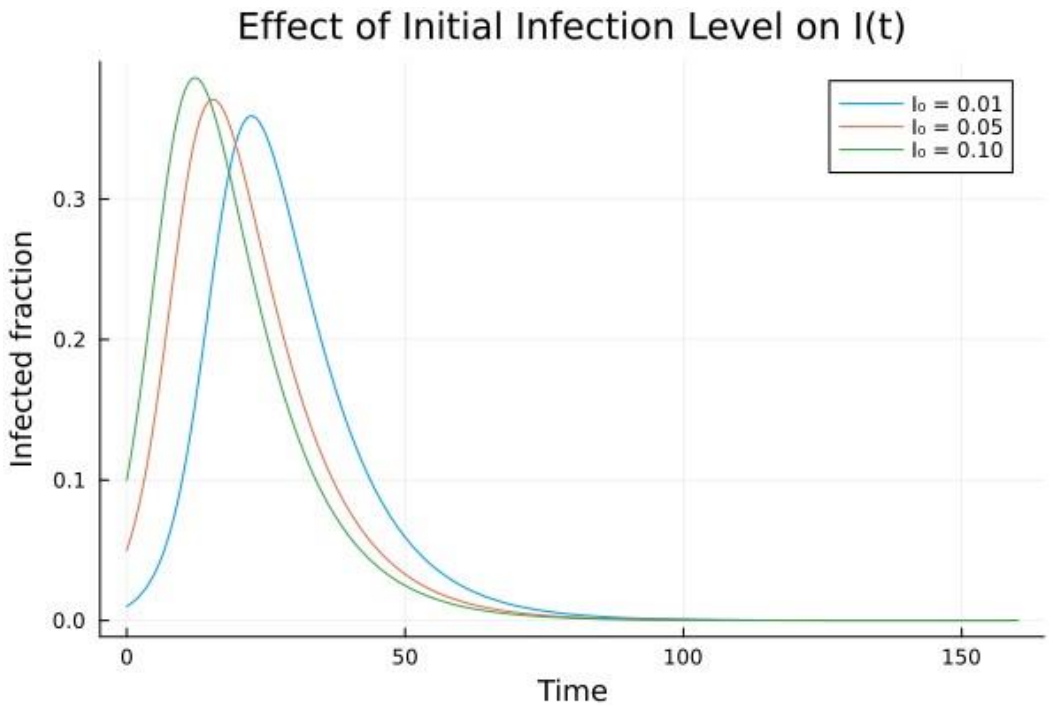


Figure 10: Infected population for varying initial infection levels.

Increasing the initial infected population leads to a faster outbreak and earlier peak infection time, while the overall epidemic structure remains similar.

9.7.2 Effect of Basic Reproduction Number

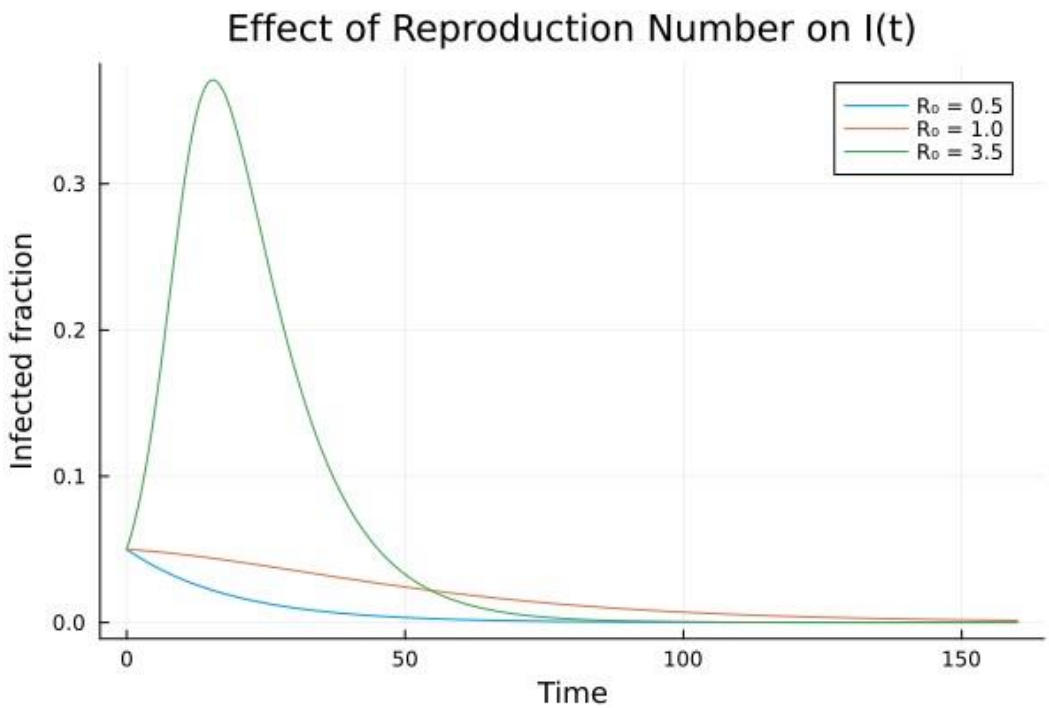


Figure 11: Infected population for varying R_0.

For $R_0 < 1$, infection decays without a significant outbreak. For $R_0 > 1$, a clear epidemic peak develops, confirming the theoretical threshold behaviour.

9.7.3 Numerical Observations

Across all simulations:

- RK4 produced smooth and stable solutions
- No non-physical oscillations or negative values were observed

9.8 Validation of Conservation Law

The conservation condition

$$S(t) + I(t) + R(t) = 1$$

was satisfied with deviations less than 10^{-12}, confirming numerical correctness.

9.9 Implementation Considerations

- Preallocation of arrays improved computational efficiency
- Explicit time-stepping ensured clarity and control
- Consistent step sizes enabled fair comparison

9.10 Appendix Summary

- Numerical methods were correctly implemented and verified
- Convergence behaviour matches theoretical predictions
- RK4 provides superior accuracy and stability
- SIR simulations confirm expected epidemiological behaviour